

BCC1313 ALGORITHMS AND COMPLEXITY

Sorting Algorithms

INTRODUCTION

- **Sorting** in data structures is the process of organizing elements in a specific order, typically **ascending** or **descending**.

- Sorting enhances the efficiency of **searching**, **analyzing**, and **managing** data.

SORTING

- There are two main categories of sorting algorithms:
  - **Comparison-Based Sorting:** Algorithms that compare elements to determine their order - Bubble Sort, Selection Sort, Merge Sort, and Quick Sort.
  - **Non-Comparison-Based Sorting:** Algorithms that use alternative methods, like counting or bucketing, and often achieving faster sorting times under certain conditions - Counting Sort, Radix Sort, and Bucket Sort.

SORTING

Key properties of sorting algorithms:

- **Stability**
  - A sorting algorithm is stable if it preserves the relative order of elements with equal keys (values). If two elements with the same key appear in a specific order, they will remain in that order.

Input: **[(Alice, 85), (Bob, 75), (Charlie, 85)]**

Sorting by marks (ascending):

[(Bob, 75), (Alice, 85), (Charlie, 85)]

SORTING

- **In-Place Sorting:** sorts the data directly in the input array or list, without requiring additional storage or memory for a second copy of the data. Example:

  Input order: **[5, 3, 8, 6, 2]**

  Sorted Result: **[2, 3, 5, 6, 8]**

APPLICATION OF SORTING

- Sorting is essential in various real-world applications:
  - **Database Management:** Sorting helps optimize searching and querying operations, making data retrieval faster and more efficient.
  - **Search Algorithms:** Many search algorithms, such as binary search, require the data to be sorted for optimal performance.

APPLICATION OF SORTING

- **Data Analysis:** Sorting helps organize data for tasks like finding medians, modes, or top-k elements.
- **Computer Graphics:** Sorting is used in tasks like object ordering in rendering process.
- **Machine Learning:** Sorting plays a key role in data pre-processing and feature selection to enhance model accuracy and efficiency.

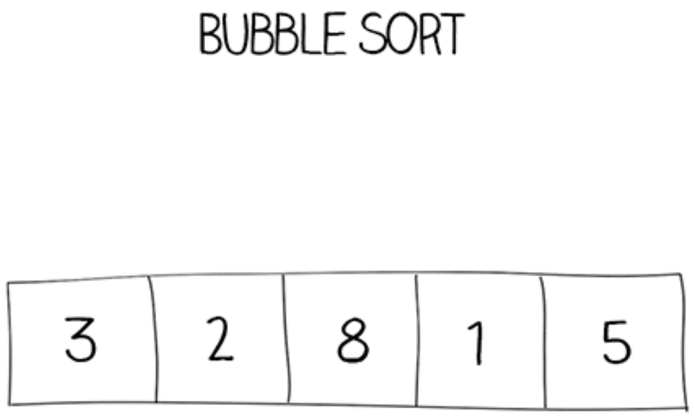
SORTING ALGORITHMS

- Sorting is a process of arranging a group of data in either ascending or descending order.

- There are several sorting algorithms such as:
  - **Bubble sort**
  - **Selection Sort**
  - **Insertion Sort**
  - **Quick Sort**
  - **Merge Sort**

BUBBLE SORT

- **Bubble sort** is the simplest and most straightforward sorting algorithm. It is best suited for sorting small datasets, as it has a relatively high time complexity, making it inefficient for larger datasets.



BUBBLE SORT

Algorithms:

1. Comparison and Swapping

- The algorithm begins by comparing the first two elements of the list.
- If the first element is greater than the second, they are swapped.
- If the first element is smaller than or equal to the second, no swap is made.

BUBBLE SORT

2. Iteration

- The algorithm then moves to the next pair of elements and repeats the comparison and swapping process.
- This process is repeated for each pair of adjacent elements in the list.

BUBBLE SORT

3. Passes

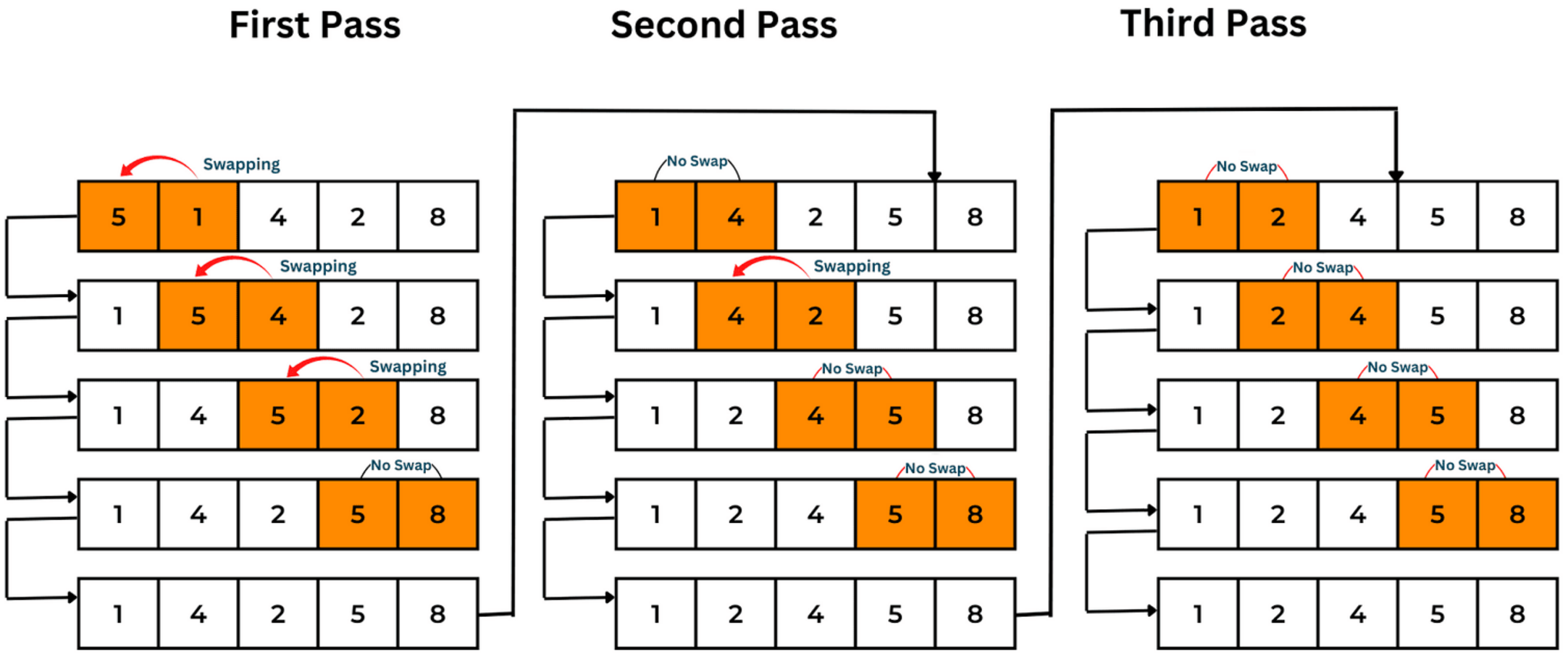
- After the first pass, the largest element has moved to its correct position at the end of the list.
- The algorithm then repeats the process for the remaining elements (excluding the last one) in the second pass.
- The process continues until no more swaps are needed in a pass, indicating that the list is sorted.

BUBBLE SORT

4. Optimization

- It’s important to note that the algorithm can be optimized by stopping the algorithm if no swaps are made during a pass, as this indicates that the list is already sorted.

BUBBLE SORTING



```
void bubbleSort(int array[]) {
    int size = 5;
    int temp;
    bool swapped;
    for (int path = 1; path <= size - 1; path++) {
        swapped = false; for (int x = 0; x < size - path; x++) {
            if (array[x] > array[x + 1]) {
                temp = array[x];
                array[x] = array[x + 1];
                array[x + 1] = temp;
                swapped = true;
            }
        }
        if (!swapped) {
            break;
        }
    }
}
```

ANALYSIS OF BUBBLE SORT

- **Time Complexity**
  - The **worst-case** time complexity of the optimized Bubble Sort is **$O(n^2)$,** which is the same as the standard version. This occurs when the algorithm has to compare and swap every pair of elements.
  - However, the optimization helps in cases where the array is already or nearly sorted.
  - In the **best case** (sorted array), the time complexity can be **O(n)** since the algorithm stops early when no swaps are made.

ANALYSIS OF BUBBLE SORT

- **Space Complexity**
  - Bubble Sort has a space complexity of **O(1),** meaning it uses a constant amount of extra space. It sorts the array in place, so no additional memory is needed.
- **Stability**
  - Bubble Sort is stable - it keeps the relative order of equal elements intact. If two elements are equal and do not need to be swapped, their order remains unchanged.

ANALYSIS OF BUBBLE SORT

- **Stability**
  - Bubble Sort is a stable sorting algorithm. Stability means that the relative order of equal elements remains unchanged after sorting.
  - In Bubble Sort, if two elements have the same value, and the algorithm does not swap them because they are in the correct order, their relative order will be preserved.

PRACTICE

1. Sort the following unsorted array using the Bubble Sort algorithm:

[27, 80, 2, 46, 16, 12, 50]

SOLUTION

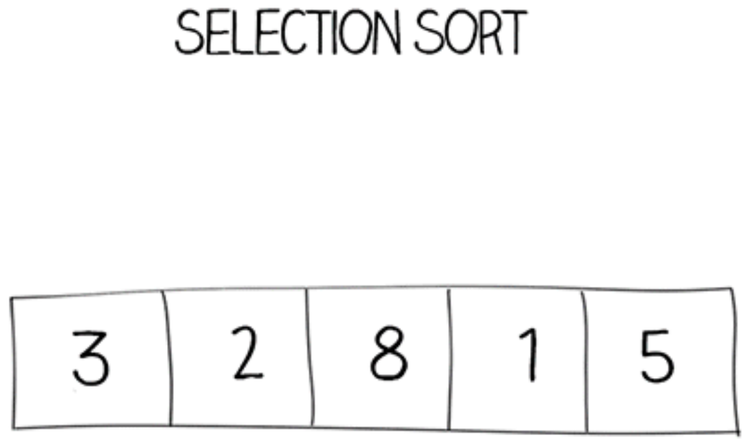
Pass	Steps	Array
Initial	Original array	[27, 80, 2, 46, 16, 12, 50]
1	Compare 27 > 80: No swap Compare 80 > 2: Swap Compare 80 > 46: Swap Compare 80 > 16: Swap Compare 80 > 12: Swap Compare 80 > 50: Swap	[27, 80, 2, 46, 16, 12, 50] [27, 2, 80, 46, 16, 12, 50] [27, 2, 46, 80, 16, 12, 50] [27, 2, 46, 16, 80, 12, 50] [27, 2, 46, 16, 12, 80, 50] [27, 2, 46, 16, 12, 50, 80]
2	Compare 27 > 2: Swap Compare 27 > 46: No swap Compare 46 > 16: Swap Compare 46 > 12: Swap Compare 46 > 50: No swap	[2, 27, 46, 16, 12, 50, 80] [2, 27, 46, 16, 12, 50, 80] [2, 27, 16, 46, 12, 50, 80] [2, 27, 16, 12, 46, 50, 80] [2, 27, 16, 12, 46, 50, 80]

SOLUTION

Pass	Steps	Array
3	Compare 2 > 27: No swap Compare 27 > 16: Swap Compare 27 > 12: Swap Compare 27 > 46: No swap	[2, 27, 16, 12, 46, 50, 80] [2, 16, 27, 12, 46, 50, 80] [2, 16, 12, 27, 46, 50, 80] [2, 16, 12, 27, 46, 50, 80]
4	Compare 2 > 16: No swap Compare 16 > 12: Swap Compare 16 > 27: No swap	[2, 16, 12, 27, 46, 50, 80] [2, 12, 16, 27, 46, 50, 80] [2, 12, 16, 27, 46, 50, 80]
5	Compare 2 > 12: No swap Compare 12 > 16: No swap	[2, 12, 16, 27, 46, 50, 80] [2, 12, 16, 27, 46, 50, 80]

SELECTION SORT

- **Selection Sort** works by repeatedly selecting the smallest element from the unsorted portion of the list and moving it to the front.



SELECTION SORT

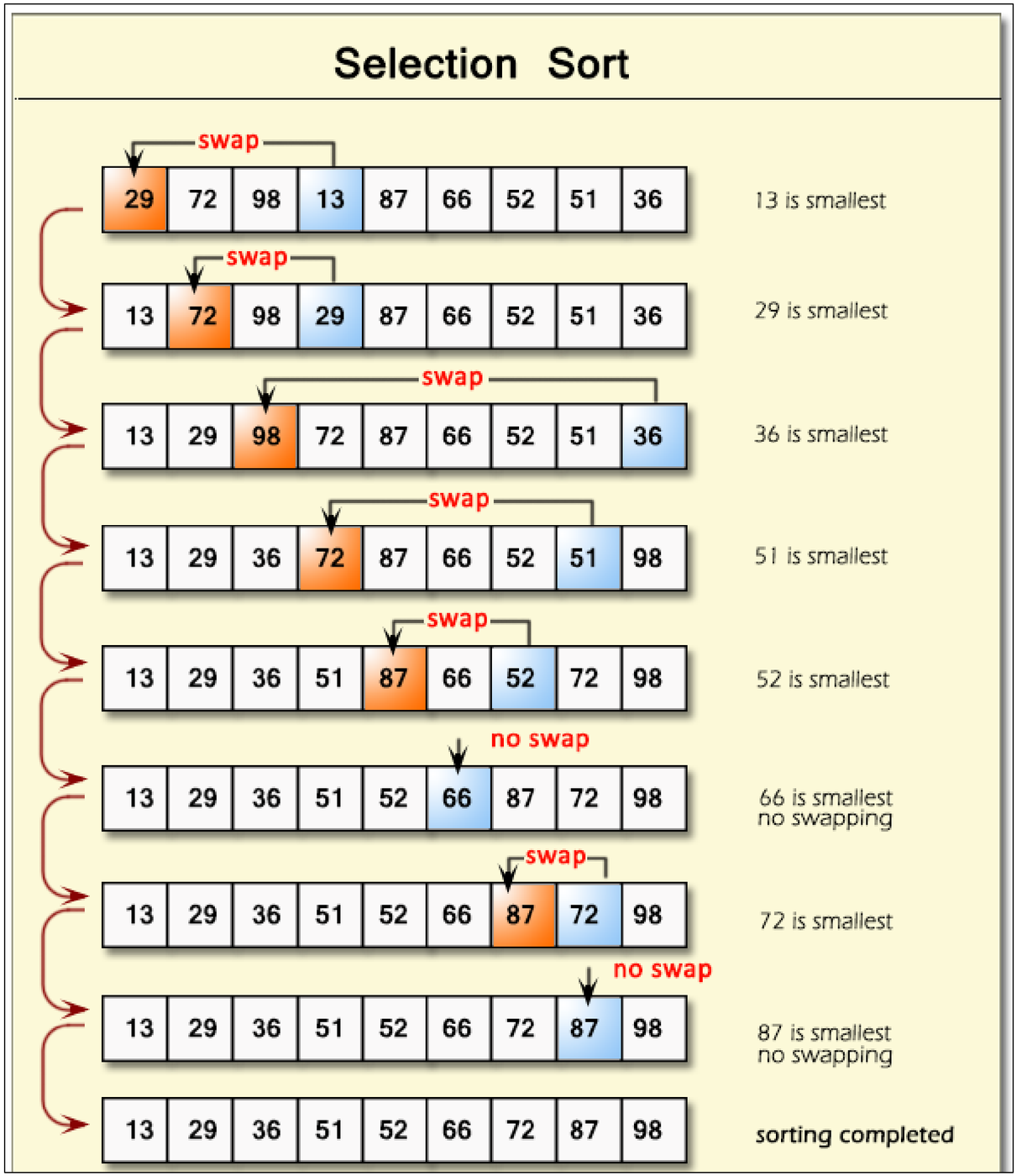
Algorithms:

1. **Start with the entire list -** Begin with the entire list unsorted.
2. **Find the smallest element -** Look through the unsorted list (starting from the first element) to find the smallest element.
3. **Swap with the first element -** Once the smallest element is found, swap it with the first element of the unsorted portion.

SELECTION SORT

4. **Move to the next element -** ignore the first element and repeat the process with the remaining unsorted portion of the list.
5. **Repeat -** Continue the process until the entire list is sorted. After each pass, the unsorted portion of the list gets smaller, and the sorted portion grows larger.

Selection Sort



```
void selectionSort(int arr[], int n)
{
     for (int i = 0; i < n - 1; ++i)
     {
          int minIndex = i;
          for (int j = i + 1; j < n; ++j)
          {
               if (arr[j] < arr[minIndex])
               {
                    minIndex = j;
               }
               if (minIndex != i)
               {
                    swap(arr[i], arr[minIndex]);
               }
          }
}
```

PRACTICE

1. Sort the following unsorted array using the Selection Sort algorithm:

[27, 80, 2, 46, 16, 12, 50]

SOLUTION

	Unsorted
	Sorted

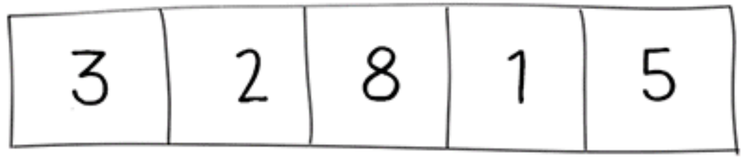
Pass	Steps	Array
Initial	Original array	[27, 80, 2, 46, 16, 12, 50]
1	Swap smallest (2) with 27	[2, 80, 27, 46, 16, 12, 50]
2	Swap smallest (12) with 80	[2, 12, 27, 46, 16, 80, 50]
3	Swap smallest (16) with 27	[2, 12, 16, 46, 27, 80, 50]
4	Swap smallest (27) with 46	[2, 12, 16, 27, 46, 80, 50]
5	Smallest element (46) is already in position	[2, 12, 16, 27, 46, 80, 50]
6	Swap smallest (50) with 80	[2, 12, 16, 27, 46, 50, 80]

INSERTION SORT

- **Insertion Sort** is a simple sorting algorithm that builds the final sorted array one element at a time.
- It works much like sorting playing cards

  - pick up cards one by one and insert each one into its correct position in the already sorted part of the deck.





INSERTION SORT

Algorithms:

1. **Start with the entire list -** Begin with the entire list unsorted.
2. **Find the smallest element -** Look through the unsorted list (starting from the first element) to find the smallest element.
3. **Swap with the first element -** Once the smallest element is found, swap it with the first element of the unsorted portion. This places the smallest element at the beginning of the list

```
void insertion(int array[])
{
     int item,path,insertionPoint,size=5;

     for (path=1; path<size; path++)
     {
          item=array[path];
          insertionPoint=path;

          while ((insertionPoint>0) &&(array[insertionPoint-1] >item))
          {
               array[insertionPoint]=array[insertionPoint-1];
               insertionPoint--;
          }
          array[insertionPoint]=item;

     }
}
```

PRACTICE

1. Sort the following unsorted array using the Insertion Sort algorithm:

[27, 80, 2, 46, 16, 12, 50]

SOLUTION

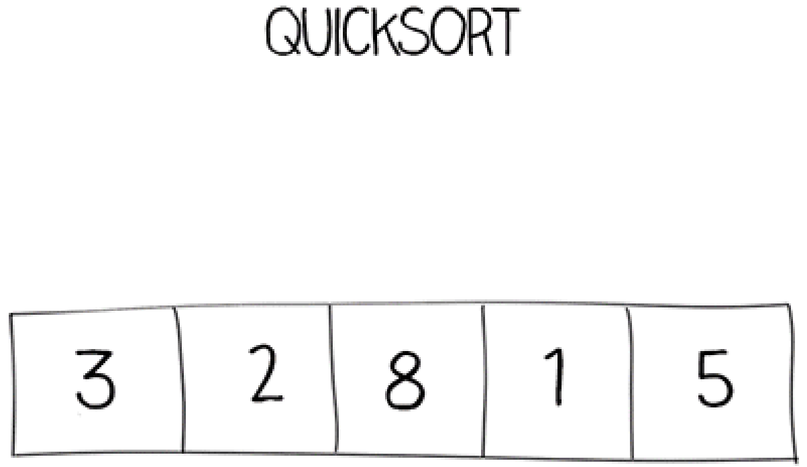
Pass	Key Element	Steps	Array
Initial	-	Original array	[27, 80, 2, 46, 16, 12, 50]
1	80	Compare 80 with 27: No change (80 > 27)	[27, 80, 2, 46, 16, 12, 50]
2	2	Compare 2 with 80: Shift 80 Compare 2 with 27: Shift 27 Insert 2	[2, 27, 80, 46, 16, 12, 50]
3	46	Compare 46 with 80: Shift 80 Compare 46 with 27: No change Insert 46	[2, 27, 46, 80, 16, 12, 50]
4	16	Compare 16 with 80: Shift 80 Compare 16 with 46: Shift 46 Compare 16 with 27: Shift 27 Insert 16	[2, 16, 27, 46, 80, 12, 50]

SOLUTION

Pass	Key Element	Steps	Array
5	12	Compare 12 with 80: Shift 80 Compare 12 with 46: Shift 46 Compare 12 with 27: Shift 27 Compare 12 with 16: Shift 16 Insert 12	[2, 12, 16, 27, 46, 80, 50]
6	50	Compare 50 with 80: Shift 80 Compare 50 with 46: No change Insert 50	[2, 12, 16, 27, 46, 50, 80]

QUICK SORT

- **Quick Sort** is an efficient sorting algorithm that follows a divide-and-conquer approach.



QUICK SORT

- It consists of two main phases:
  - **Partition Phase:** In this phase, the array is divided into two smaller partitions based on a selected "pivot" element. All elements less than the pivot are moved to one partition, and all elements greater than the pivot are moved to another partition.
  - **Sort Phase:** After partitioning, the algorithm recursively applies Quick Sort to the two smaller partitions, sorting them independently

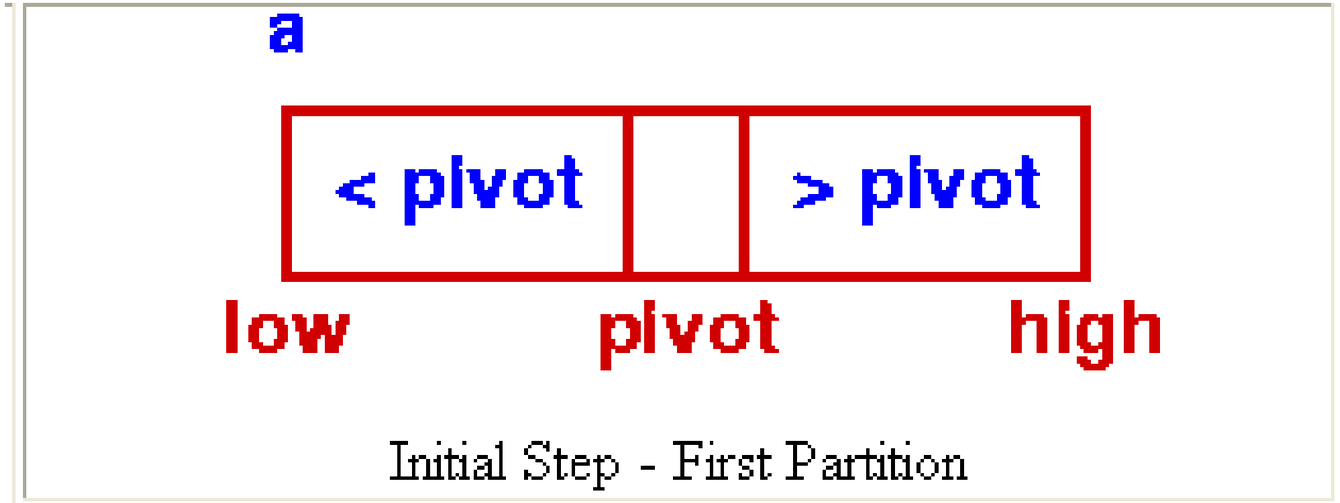
QUICK SORT

- Algorithms:
  1. **Select a Pivot**: Choose an element from the array as the pivot. Different strategies can be used to pick the pivot (e.g., first element, last element, middle element, or a random element).
  2. **Partitioning**: Rearrange the array such that all elements smaller than the pivot are placed to its left, and all elements greater than the pivot are placed to its right.

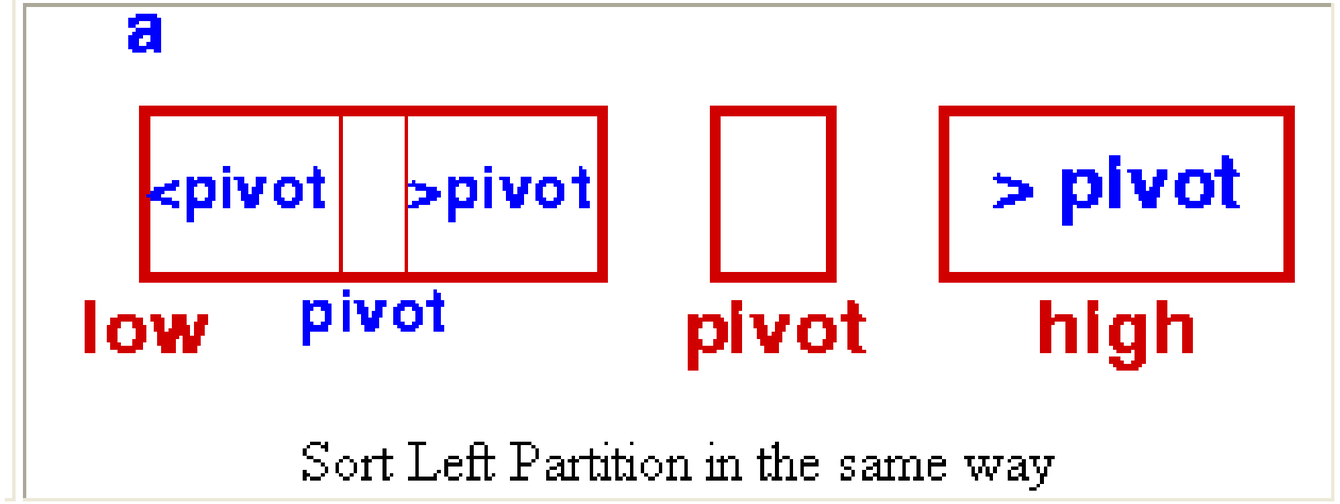
QUICK SORT

3. **Recursion**: Recursively apply the same procedure to the left and right sub-arrays (elements to the left of the pivot and elements to the right of the pivot).
4. **Base Case**: The recursion terminates when the sub-array has fewer than two elements.

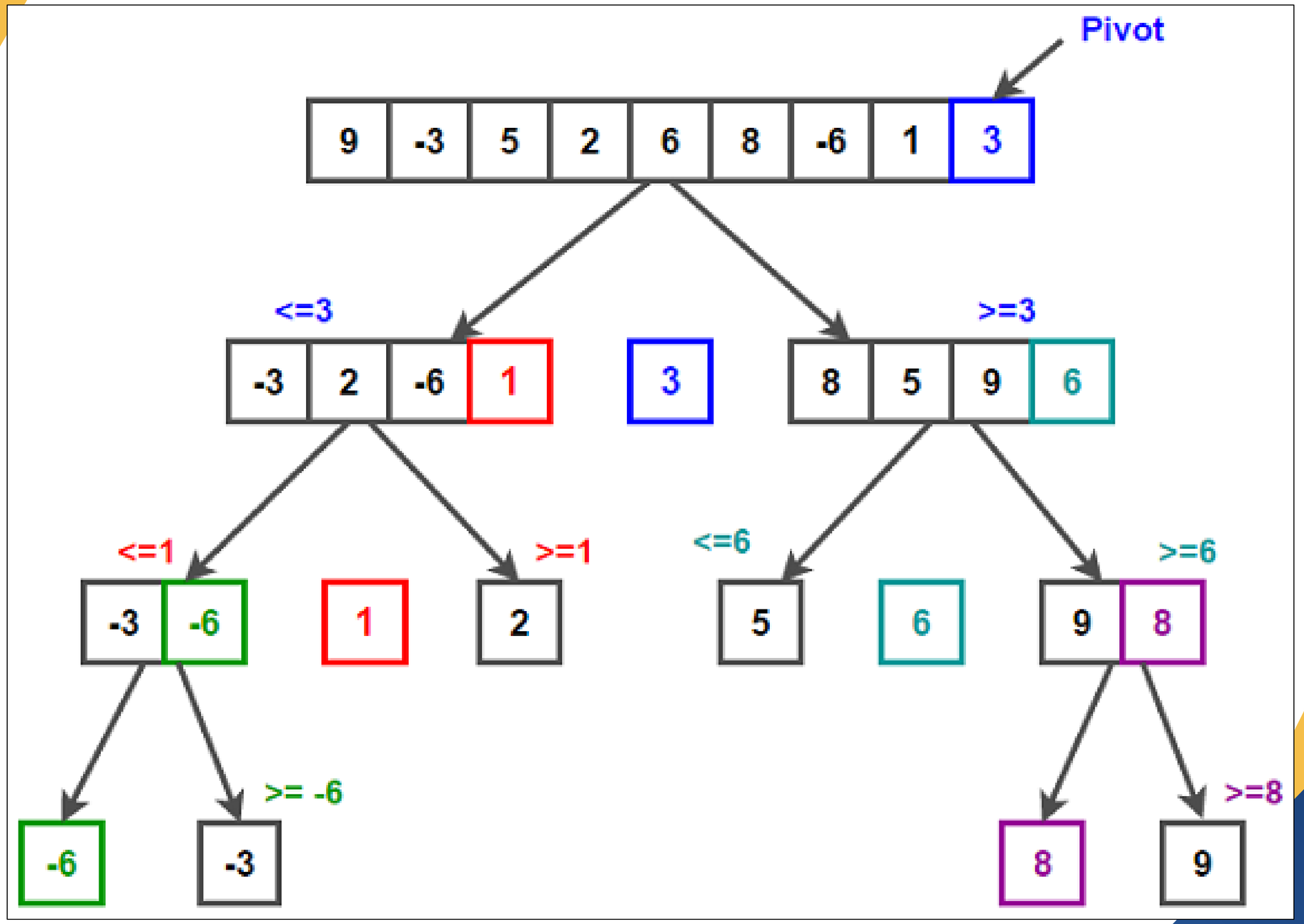
QUICK SORT



Initial Step - First Partition



Sort Left Partition in the same way


Pivot
9 -3 5 2 6 8 -6 1 3
<=3
>=3
-3 2 -6 1
3
8 5 9 6
<=1
>=1
<=6
>=6
-3 -6
1
2
5
6
9 8
>= -6
>=8
-6
-3
8
9

PRACTICE

1. Sort the following unsorted array using the Quick Sort algorithm:

   Unsorted Array: [27, 80, 2, 46, 16, 12, 50]

SOLUTION

Unsorted Array: [27, 80, 2, 46, 16, 12, 50]

Step 1: Choose a pivot element

E.g. the last element in the array as the pivot. So, the pivot is 50.

Step 2: Partition the array around the pivot

- Reorder the array so that all elements smaller than 50 are on the left, and elements greater than 50 are on the right.
- After partitioning around 50:

[27, 2, 46, 16, 12, 50, 80]

SOLUTION

Step 3: Recursively apply Quick Sort to the left and right subarrays.

- Left Subarray: [27, 2, 46, 16, 12]
- Right Subarray: [80] (already sorted, so no further action needed)

Step 4: Apply Quick Sort to the left subarray [27, 2, 46, 16, 12].

- Choose the last element 12 as the pivot.
  Array before partitioning: [27, 2, 46, 16, 12]
- Partition the array around pivot 12.
  After partitioning around 12: [2, 12, 46, 16, 27]
  Now, 12 is in its correct position.

SOLUTION

Step 5: Recursively apply Quick Sort to the left and right subarrays of [2, 12, 46, 16, 27].

- Left Subarray: [2] (already sorted)
- Right Subarray: [46, 16, 27]

Step 6: Apply Quick Sort to the subarray [46, 16, 27].

- Choose the last element 27 as the pivot.

  Array before partitioning: [46, 16, 27]

  Partition the array around pivot 27.

  After partitioning around 27: [16, 27, 46]

SOLUTION

Step 7: Recursively apply Quick Sort to the left and right subarrays of [16, 27, 46].

- Left Subarray: [16] (already sorted)
- Right Subarray: [46] (already sorted)

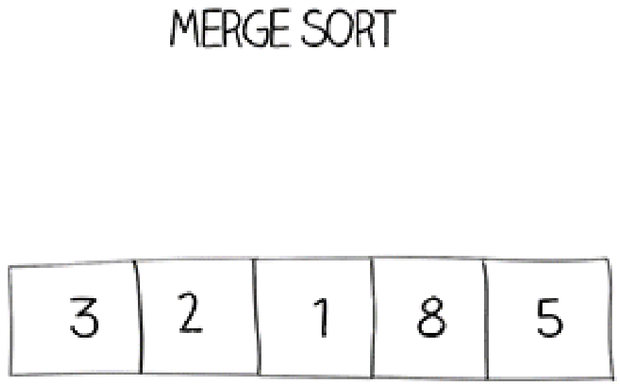
Step 8: Combine all the sorted subarrays.

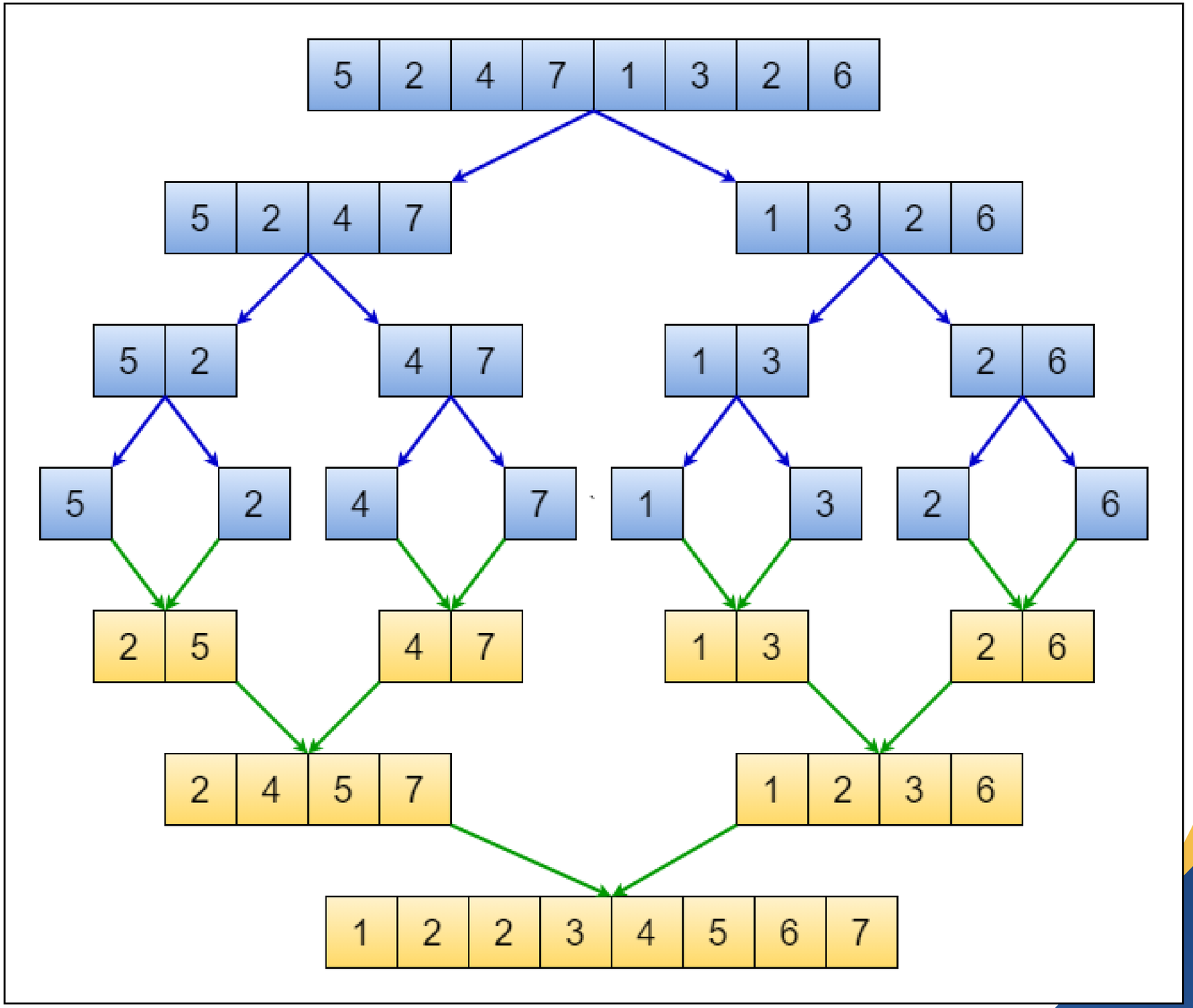
- After all the recursive calls, the final sorted array is:

Sorted Array: [2, 12, 16, 27, 46, 50, 80]

MERGE SORT

- **Merge Sort** is a divide and conquer sorting algorithm. It works by breaking the array into smaller sub-arrays, sorting them, and then merging the sorted sub-arrays to produce the final sorted array.




5 2 4 7 1 3 2 6
5 2 4 7
1 3 2 6
5 2
4 7
1 3
2 6
5
2
4
7
1
3
2
6
2 5
4 7
1 3
2 6
2 4 5 7
1 2 3 6
1 2 2 3 4 5 6 7

MERGE SORT

- Algorithms:

1. **Base case**: If the array has one or zero elements, it is already sorted.
2. **Recursive step**: Otherwise, split the array into two halves, sort each half recursively, and merge the two sorted halves.

MERGE SORT

- This algorithm is known for its efficiency and predictable O(n log n) time complexity in all cases (best, worst, and average).
- **Time Complexity:**
  - Best, worst, and average case: **O(n log n)** — Merge sort performs consistently in O(n log n) time.
- **Space Complexity**: **O(n)** — Merge Sort requires extra space for the temporary arrays used in the merging process.

MERGE SORT

Why Merge Sort is Efficient?

- **Stable Sorting:** Merge Sort is a stable algorithm, meaning it preserves the relative order of equal elements.
- **Divide and Conquer:** The algorithm divides the problem into smaller sub-problems, making it easier to solve.
- **Consistent Performance:** Merge Sort has predictable time complexity of O(n log n), which makes it efficient even for large data sets.

SUMMARY

- Sorting is the process of arranging data in a specific order (usually ascending or descending) to facilitate efficient searching, processing, and management.
- Sorting algorithms fall into two main categories:
  - Comparison-Based Sorting
  - Non-Comparison-Based Sorting